

AI Fundamentals Project Portfolio

Search, Learning, Decision Making, Games, and Logic-Based Reasoning

Fundamentals and Applications of Artificial Intelligence

June 5, 2026

Abstract

This report documents a consolidated portfolio of five course projects in artificial intelligence. The projects cover classical state-space search, machine learning with gradient-based optimization, local search, Markov decision processes, reinforcement learning, adversarial game playing, first-order logic inference, and hybrid LLM-symbolic reasoning. The repository was reorganized into a clean phase-based structure while preserving the source code, notebooks, runtime assets, and executable entry points required for each assignment.

Contents

1	Repository Overview	2
2	Phase 1: Search, Regression, and Local Search	2
2.1	State-Space Search	2
2.2	Linear Regression	2
2.3	DNA Center Finding	3
3	Phase 2: MDP and Reinforcement Learning	3
3.1	Known Environment: Value Iteration	3
3.2	Unknown Environment: Q-Learning	3
3.3	Bonus: Deep Q Network	3
4	Phase 3: Adversarial Search Game	3
5	Phase 4: First-Order Logic Minesweeper	4
6	Phase 5: LLM and Symbolic Mastermind	4
7	Engineering Cleanup	4
8	Execution Notes	5
9	Conclusion	5

1 Repository Overview

The final repository is organized as a single portfolio with independent project phases under `projects/`. Each phase keeps its original executable surface while removing classroom meta-data, temporary clone directories, generated outputs, caches, and hardcoded secrets.

Phase	Directory	Core Topic
1	<code>phase-01-search-regression-annealing</code>	Search, regression, local search
2	<code>phase-02-mdp-reinforcement-learning</code>	MDP, Q-learning, DQN
3	<code>phase-03-adversarial-search-game</code>	Adversarial search
4	<code>phase-04-fol-minesweeper</code>	Logical inference
5	<code>phase-05-llm-fol-mastermind</code>	LLM and symbolic reasoning

Table 1: Consolidated project structure.

2 Phase 1: Search, Regression, and Local Search

2.1 State-Space Search

The first search task models an Angry Birds-inspired grid environment. The agent must navigate a map, collect targets, avoid collisions, and reason about enemy movement. Four search strategies are implemented:

- **Breadth-First Search (BFS):** explores by path depth and returns a shortest action sequence in uniform step settings.
- **Uniform Cost Search (UCS):** prioritizes cumulative path cost and handles non-uniform movement costs.
- **Depth-Limited / Iterative Deepening Search:** controls memory use by repeatedly searching with increasing depth bounds.
- **A* Search:** combines path cost with a heuristic based on weighted distance, obstacles, target coverage, and enemy proximity.

The search implementations use an expanded state key that includes the agent position, remaining targets, and the enemy cycle step. This prevents incorrect pruning when a position is safe at one time step but unsafe at another.

2.2 Linear Regression

The regression notebook focuses on asteroid diameter prediction. It includes preprocessing, feature scaling, exploratory analysis, and evaluation using standard regression metrics such as:

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad (1)$$

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \quad (2)$$

The main educational value is implementing stochastic gradient descent from scratch rather than relying only on a library estimator. The notebook also introduces regularization, validation splits, learning-rate behavior, and model evaluation.

2.3 DNA Center Finding

The DNA center task is a closest-string optimization problem. Given a set of DNA strings of equal length, the objective is to find a candidate string that minimizes the maximum Hamming distance to all strings:

$$\min_s \max_{x \in X} d_H(s, x) \quad (3)$$

The notebook compares greedy hill climbing, simulated annealing, and brute-force baselines. Simulated annealing is useful because it can accept worse intermediate states with a probability controlled by temperature, helping the search escape local minima.

3 Phase 2: MDP and Reinforcement Learning

3.1 Known Environment: Value Iteration

The MDP project uses a stochastic WALL-E grid world. The agent collects trash, avoids enemies and buildings, manages power thresholds, and reaches the final plant. The known-environment solver builds reward maps and applies value iteration:

$$V_{k+1}(s) = \max_a \sum_{s'} P(s' | s, a) [R(s, a, s') + \gamma V_k(s')] \quad (4)$$

The implementation precomputes policies for goal-seeking, trash collection, and threshold-specific objectives. It also records convergence history and exports heat maps for policy analysis.

3.2 Unknown Environment: Q-Learning

The unknown-environment task removes access to the transition and reward models. The agent learns an action-value function through interaction:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right] \quad (5)$$

The state representation includes position, power level, and a bitmask of remaining trash. Epsilon-greedy exploration balances discovery and exploitation, while parameter decay gradually shifts the policy toward learned behavior.

3.3 Bonus: Deep Q Network

The bonus DQN implementation replaces the Q-table with a neural network approximation. It includes replay memory, a target network, and a dueling architecture that separates state value and action advantage. This is more scalable than tabular Q-learning, but it requires additional compute and the PyTorch dependency.

4 Phase 3: Adversarial Search Game

The Paint Battle project introduces a competitive two-player grid game. The implemented AI uses minimax search with alpha-beta pruning:

$$\alpha = \max(\alpha, v), \quad \beta = \min(\beta, v) \quad (6)$$

Branches are pruned when $\alpha \geq \beta$, reducing the number of explored game states without changing the final minimax decision. The evaluation function considers score difference, territory control, mobility, player survival, distance pressure, and local tactical opportunities. Move

ordering and cached state evaluation are used to improve practical performance at deeper search depths.

5 Phase 4: First-Order Logic Minesweeper

The Minesweeper solver represents the board as logical facts:

- `hidden(r,c)` for unrevealed cells.
- `revealed(r,c)` for known safe cells.
- `flagged(r,c)` for inferred mines.
- `clue(r,c,v)` for numeric revealed clues.
- `neighbor(r,c,nr,nc)` for board topology.

Two central inference rules drive the solver. If the number of flagged neighbors equals the clue, all remaining hidden neighbors are safe. If the number of flagged plus hidden neighbors equals the clue, all remaining hidden neighbors are mines. When deterministic inference is exhausted, the solver estimates local risk and selects a low-probability hidden cell.

6 Phase 5: LLM and Symbolic Mastermind

The Mastermind project combines natural-language model output with symbolic verification. The game provides feedback as correct, misplaced, and incorrect counts. The solver creates a Prolog knowledge base for valid codes and score calculation. After each guess, an LLM proposes a Prolog constraint of the form:

```
turn_1(Candidate) :- calc_score(Candidate, ['A','B','C'], 1, 1, 1).
```

The Python controller validates the generated rule before asserting it into Prolog. This design treats the LLM as a rule-generation assistant, while Prolog remains responsible for deterministic candidate filtering. The cleaned repository reads API settings from environment variables rather than embedding credentials in source code.

7 Engineering Cleanup

The consolidation applied the following cleanup rules:

- Preserve executable source code, notebooks, maps, icons, and required binary agents.
- Remove clone metadata, classroom-only badges, generated plots, screenshots, caches, and temporary files.
- Add one root dependency file and clear run instructions.
- Replace hardcoded LLM credentials with environment variables.
- Add a root README and phase-level README files.
- Add a LaTeX report suitable for rendering into a final PDF.

8 Execution Notes

Most scripts should be run from their own phase directory because several projects use relative paths for images, maps, generated plots, or Prolog files. The LLM Mastermind solver requires an OpenAI-compatible API key and SWI-Prolog installed on the host machine. The DQN bonus requires PyTorch and can take significantly longer than the other scripts.

9 Conclusion

Together, the five phases form a coherent AI fundamentals portfolio. The work begins with deterministic search and optimization, moves into probabilistic decision making and reinforcement learning, extends to adversarial reasoning, and ends with symbolic and hybrid LLM-symbolic systems. The consolidated repository keeps these projects independent enough to run and inspect, while presenting them under a single professional structure.